EXPERIMENT – 7

Title: Implementation of Stop & Wait Mechanism

Objective:

To understand and implement stop and wait flow control mechanism in data communication.

Software / Tools Required

- Operating System: Linux
- Programming Language: Python 3
- Custom Module: netpack

Theory

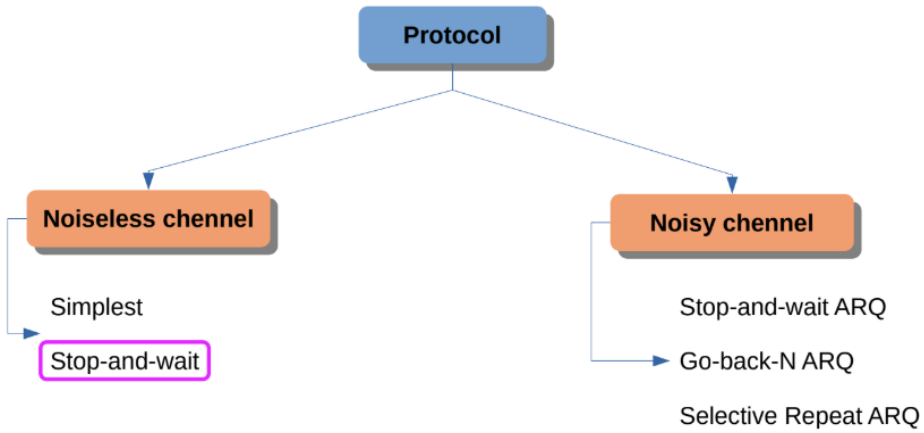
Stop-and-wait is a simple flow control protocol used in the data link layer. In this method, the sender sends one frame and then waits for an acknowledgment from the receiver before sending the next frame.

If the acknowledgment is received, the sender sends the next frame. If not, it waits until the acknowledgment arrives. This ensures that data is transmitted in an orderly and reliable way.

This protocol works well in a **noiseless channel**, where frames and acknowledgments are not lost or corrupted.

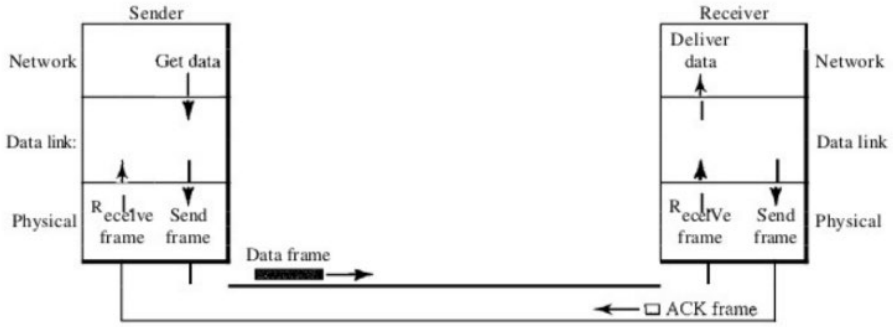
In this experiment, the sender sends data frame by frame and the receiver sends an **ACK** for each frame. After receiving the ACK, the sender proceeds to send the next frame.

Protocol Concept

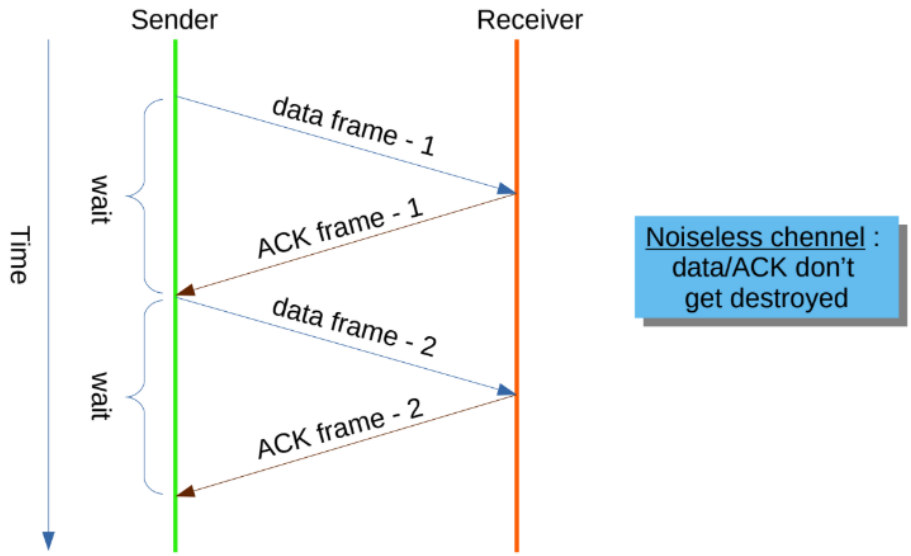


Stop-and-Wait Protocol

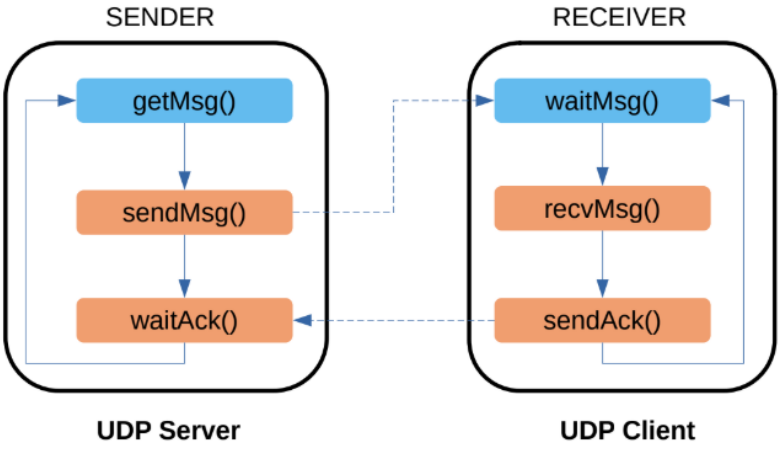
The sender sends one frame, stops until it receives confirmation from the receiver (okay to go ahead), and then sends the next frame.



Stop-and-Wait Flow



Program Flow



Procedure / Algorithm

Sender Side

1. Import required modules and create UDP socket.
2. Wait for receiver connection.
3. Send one frame and wait for ACK.
4. If ACK is received, send next frame.
5. After sending all data, send termination signal.

Receiver Side

1. Import required modules and create UDP socket.
2. Connect to sender.
3. Receive frame and display it.
4. Send ACK for each received frame.
5. Stop communication when termination signal is received.

Program Code

Sender Program:

```
import netpack as npk
import stop_wait as smn

# Settings:
PORT=9090
SERVER="127.0.0.1"
ADDR=(SERVER,PORT)


# Step-1:
tx=npk.create_udp_socket()

# Step-2:
npk.start_udp_server(tx, ADDR)

# Waiting for Rx to connect:
print('-'*50+'\n Tx is ready to accept client ...')

# Capture "Hello" message from client:
while True:
    (msg, addr)=npk.read_data_from_udp_client(tx)
    if msg=='Hello':
        print('[Connected] Client : {}'.format(addr))
        break

# Invoke sender method:
smn.sender(tx, addr)

# Close socket object:
npk.close_socket_connection(tx)
print('[SENDER] Process is finished')
```

Receiver Program:

```python
import netpack as npk
import stop_wait as smn


# Settings:
PORT=9090
SERVER="127.0.0.1"
ADDR=(SERVER,PORT)

# Ste-1
rx=npk.create_udp_socket()

print('[=] Client is connecting to IP & PORT at Server side {}'.format(ADDR))

# Connect client socket object to PORT with hello message:
npk.send_data_to_udp_server(rx, "Hello", ADDR)

# Invoke reciever method:
smn.receiver(rx, ADDR)

# Close socket object:
npk.close_socket_connection(rx)
```

Stop & Wait Module

```python
'''
This module contains stop & wait protocol methods
'''

import time
import netpack as npk

data='RCCIIT ECE'
frame_data=[i for i in data]
ack_data='ACK'

# Sender Method:
def sender(tx_socket,rx_addr):
    global frame_data
    global ack_data

    for msg in frame_data:
        time.sleep(1)
        # Tx frame :
        npk.send_data_to_udp_client(tx_socket, msg, rx_addr)
        print('[SENT] frame data - [{}]'.format(msg))

        # Wait to Rx ACK frame:
        while True:

            (msg_rx, addr)=npk.read_data_from_udp_client(tx_socket)
            if msg_rx==ack_data:
                print('[ACK] Recieved for - [{}]'.format(msg))
                break

    # Tx NULL character to end connection:
    npk.send_data_to_udp_client(tx_socket,'\0',rx_addr)

    print('Terminate communication.')
    return


# Receiver Method:
def receiver(rx_socket, tx_ADDR):
    global ack_data
    rx_frame=[]

    while True:
        # Rx frame :
        (msg_rx, ADDR)=npk.read_data_from_udp_server(rx_socket)

        if msg_rx=='\0':
            print('Communication terminated')
            break
        else:
            rx_frame.append(msg_rx)
        print('[RECV] frame data - [{}]'.format(msg_rx))

        # Tx ACK frame :
        npk.send_data_to_udp_server(rx_socket, ack_data, tx_ADDR)
        print('[SENT] Ack frame')

    print("[=] Received Data :")
    for i in rx_frame:
        print('{}'.format(i),end='')
    print("\n")
    return
```

Output:

Sender Side Output

```
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-7$ python3 7_stop_wait_Tx.py
-----------------------------------------------
 Tx is ready to accept client ...
[Connected] Client : ('127.0.0.1', 55923)
[SENT] frame data - [R]
[ACK] Recieved for - [R]
[SENT] frame data - [C]
[ACK] Recieved for - [C]
[SENT] frame data - [C]
[ACK] Recieved for - [C]
[SENT] frame data - [I]
[ACK] Recieved for - [I]
[SENT] frame data - [I]
[ACK] Recieved for - [I]
[SENT] frame data - [T]
[ACK] Recieved for - [T]
[SENT] frame data - [ ]
[ACK] Recieved for - [ ]
[SENT] frame data - [E]
[ACK] Recieved for - [E]
[SENT] frame data - [C]
[ACK] Recieved for - [C]
[SENT] frame data - [E]
[ACK] Recieved for - [E]
Terminate communication.
[SENDER] Process is finished
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-7$
```

Receiver Side Output

```
sumit@VivoBook-16X:/mnt/c/Users/sumit/Documents/B TECH 3rd YEAR STUFF/6th sem/Networking/exp-7$ python3 7_stop_wait_Rx.py
[=] Client is connecting to IP & PORT at Server side ('127.0.0.1', 9090)
[RECV] frame data - [R]
[SENT] Ack frame
[RECV] frame data - [C]
[SENT] Ack frame
[RECV] frame data - [C]
[SENT] Ack frame
[RECV] frame data - [I]
[SENT] Ack frame
[RECV] frame data - [I]
[SENT] Ack frame
[RECV] frame data - [T]
[SENT] Ack frame
[RECV] frame data - [ ]
[SENT] Ack frame
[RECV] frame data - [E]
[SENT] Ack frame
[RECV] frame data - [C]
[SENT] Ack frame
[RECV] frame data - [E]
[SENT] Ack frame
Communication terminated
[=] Received Data :
RCCIIT ECE
```

Result

The stop and wait protocol was implemented successfully and data was transmitted with proper acknowledgment.

Conclusion

From this experiment we learned how stop-and-wait ensures reliable data transfer by sending one frame at a time and waiting for acknowledgment.